MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring

Final Report

Michaľ Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
office@pads.rwth-aachen.de

Abstract. Predictive process monitoring uses historical event log data to predict outcomes and remaining times of running process instances, with standard approaches deriving features from the event log alone while ignoring temporal resource-load variation. This project tests whether aligned time-series features (active-case counts with rolling-window statistics) improve prediction quality. We built a modular four-stage pipeline for the BPI Challenge 2017 loan application log, comparing log-only Random Forest models against log+time-series models for binary outcome classification (approved vs. not-approved) and remaining-time regression. The hourly-resampled time-series features with 8-hour rolling windows added no meaningful improvement for classification (F1-weighted +0.1%) and degraded regression (RMSE +1.45). We analyse why this extraction strategy fails to improve either task and discuss implications for feature engineering in predictive process monitoring using our open-source and fully reproducible pipeline.

Keywords: Predictive Process Monitoring · Process Mining · Time-Series Features · Random Forest · Model Evaluation · Explainability

1 Introduction

Predictive process monitoring (PPM) predicts future states of running process instances from event log data, with common tasks including binary outcome prediction (will this case end normally or deviate?) and remaining-time regression (how many hours until completion?). Most PPM approaches derive features from the event log only, such as activity counts, elapsed time, and case attributes, capturing what happened in a case but not the concurrent workload on the process. The BPI Challenge 2017 dataset [1] records a loan application process at a Dutch financial institution with validation, offer, and approval stages, where resource contention during peak periods can affect both outcomes and processing times. Log-based features alone do not capture this workload variation over time.

This project answers: *do aligned time-series workload features improve predictive performance for outcome and remaining-time tasks compared to event-log features alone?* We built a modular pipeline that extracts log-based fea-

tures, computes time-series features, trains Random Forest models, and evaluates them against baselines. The contributions are: an open-source, reproducible PPM pipeline with time-series support, an empirical comparison of log-only vs. log+time-series models on both tasks, and an analysis of when time-series features help and when they do not.

The paper is organised as follows. section 2 describes the dataset and preprocessing. section 3 covers feature engineering. section 4 specifies the models and evaluation protocol. section 5 presents results. section 6 describes testing evidence. section 7 discusses findings and limitations. section 8 concludes.

2 Data and Preprocessing

2.1 Event Log: BPI Challenge 2017

The BPI Challenge 2017 event log [1] records a loan application process at a Dutch financial institution, with cases describing the lifecycle of credit applications including application processing, offer creation, document validation, and customer interactions. Events belong to three categories: application state changes, offer state changes, and workflow activities performed by employees.

The dataset is widely used in predictive process monitoring as it contains rich control-flow, temporal, and financial information along with substantial variation in case duration and outcome. Table 1 summarises the dataset.

Table 1: BPI Challenge 2017 dataset statistics.

Statistic	Value
Cases	31,509
Events	1,202,267
Activities	26
Start date	2016-01-01
End date	2017-02-01
Mean case length (events)	38.16
Median case length (events)	35

2.2 Prediction Tasks

This study evaluates model performance on two predictive process monitoring tasks: loan outcome classification and remaining-time regression. Cases terminate as approved, declined, or cancelled, corresponding to the occurrence of `A_Pending`, `A_Denied`, or `A_Cancelled` respectively. For outcome prediction, we formulate a binary classification task with approved cases as the positive class while declined and cancelled cases form a single negative class. For remaining-time prediction, the target variable is the time in hours between the last event of a prefix and case completion.

2.3 Preprocessing Pipeline

Raw XES event logs are first ingested, cleaned, and sorted in ascending order by timestamp before a sliding-window approach generates training instances. For each case, a window of length k at time step t consists of the most recent k events ending at position t, with k constrained by a minimum and maximum window length. Each resulting window is associated with the corresponding case label. Table 2 summarises the final dataset sizes for each prediction task along with the applied filtering steps and sliding-window configuration.

The preprocessing pipeline differs between the regression and classification tasks. For regression, no additional filtering is applied as sliding windows are generated with a minimum length of 3 and a maximum length of 100 events per case, producing truncated event histories that capture recent temporal context. For classification, additional constraints ensure label consistency and prevent information leakage: cases without any defined outcome events are removed (98 out of 31,509 cases), and windows containing an outcome event are excluded since their inclusion would leak information about the final case outcome into the input representation. The same minimum and maximum window lengths (3 to 100 events) are used as in the regression setup.

Table 2: Preprocessing summary per task.

Component	Value
Initial dataset	31,509 cases / 1,202,267 events
Window type	Sliding history window
Window length	min = 3, max = 100
Regression filtering	None (all cases retained)
Classification filtering	No-outcome cases removed (98 excluded), outcome prefixes removed
Regression final size	31,509 cases
Classification final size	31,411 cases

2.4 Temporal Train-Test Split

We use a temporal split by case start time to prevent leakage, ensuring all prefixes from the same case remain in the same split. Cases starting before the 0.8 quantile go to training while later cases go to testing, simulating an operational setting where the model predicts on cases that start after the training period. Table 3 shows the resulting split statistics.

Table 3: Temporal split statistics per prediction task.

	Train	Test
Classification task		
Cases	22,859	6,283
Prefixes	763,417	210,096
Split timestamp	2016-10-18 11:39:41	
Regression task		
Cases	22,895	6,302
Prefixes	827,614	228,455
Split timestamp	2016-10-18 22:39:59	

3 Feature Engineering

3.1 Log-Based Features (Baseline)

Log-based features are derived purely from the event log, with Table 4 listing the feature groups. Activity counts are one-hot encoded per event type while temporal features use the timestamp of the last event in the prefix, with all features computed in vectorised form for efficiency.

Table 4: Log-based feature groups.

Group	Features	Count
Activity counts	Count per event type per prefix	26
Temporal	Elapsed time, time since last event	2
Financial	Monthly cost, number of terms	2
Waiting states	Time in waiting activities	4
Total		34

3.2 Time-Series Features (Enhanced)

Time-series features capture concurrent process workload at hourly resolution, where for every hourly base signal we compute three features: the raw value, an 8-hour trailing mean (`rolling_mean_8h`), and a trend (`trend_8h` = current value minus value 8 hours earlier). Table 5 lists the time-series feature classes.

Table 5: Time-series feature classes.

Class	Base signal	Base cols	Total
ActiveCaseCount	Cases active at prefix timestamp	1	3
FinancialVolume	Hourly sum/mean of withdrawal, offer, cost	6	18
DecisionRate	Ratio of accepted decisions per hour	1	3
Total		8	24

3.3 Feature Extraction Pipeline

Features are extracted via a modular pipeline where each feature set implements the `PrefixFeature` protocol (`name()`, `fit()`, `__call__()`, `feature_names`), and the feature extractor generates a single feature matrix by concatenating all enabled feature sets. Time-series features are precomputed once during `fit()` and aligned to each prefix at extraction time via vectorised timestamp lookup.

4 Model Specification and Evaluation Protocol

4.1 Prediction Targets

Table 6 shows the class distributions for the train and test splits at both trace and prefix level, with the distributions highly consistent between sets and indicating a stable split without noticeable sampling bias. At trace level, a moderate class imbalance towards approved cases can be observed, becoming more pronounced at prefix level where approved cases form a larger proportion of the data.

Table 6: Class distribution for traces and prefixes (train/test split).

Level	Split		Count	Percentage (%)
Traces	Train	approved	12,694	55.5%
		not approved	10,165	44.5%
	Test	approved	3,449	54.9%
		not approved	2,834	45.1%
Prefixes	Train	approved	516,099	67.6%
		not approved	247,318	32.4%
	Test	approved	141,171	67.2%
		not approved	68,925	32.8%

Table 7 shows remaining-time statistics at prefix level, with the target distribution highly right-skewed as indicated by the substantial difference between mean and median values in both train and test sets. The mean remaining time

is approximately 300 hours while the median is considerably lower (around 205 hours), indicating that many prefixes occur relatively late in the process where little time remains until case completion, whereas a smaller number of early prefixes contribute disproportionately large remaining-time values. This leads to a long upper tail in the distribution with maximum values exceeding 3000 hours in the training set, while the interquartile range further reflects this variability with 50% of prefix instances lying approximately between 40 and 480 hours remaining.

The train and test distributions are highly consistent across all statistics, suggesting that the temporal structure of prefixes is preserved across the split with no significant distribution shift.

Table 7: Distribution of remaining time (in hours) for train and test prefix sets.

Statistic	Train	Test
Prefixes	827,614	228,455
Mean	302.8	300.0
Median	205.1	208.8
Std	315.5	298.1
Min	0	0
Max	3269.0	2348.1
Q1	42.3	45.3
Q3	479.4	488.8

4.2 Models and Hyperparameter Optimisation

Four model families were compared for each task (Table 8), including tree-based ensemble methods and linear baselines that handle non-linear relationships without requiring feature scaling. All classification models use `class_weight="balanced"` to address class imbalance.

Table 8: Models and hyperparameter search spaces.

Model	Task	Key hyperparameters
Random Forest	Both	`n_estimators`: 100–300 `max_depth`: 5–20 `min_samples_split`: 2–20
Logistic Regression	Classification	`C`: 0.01–100
Ridge Regression	Regression	`alpha`: 0.01–100
HistGradientBoosting	Both	`learning_rate`: 0.01–0.3 `max_iter`: 100–300

Hyperparameters are optimised via `RandomizedSearchCV` with 5-fold cross-validation using all training prefixes for faster iteration. Table 9 lists the search configuration.

Table 9: Hyperparameter optimisation configuration.

Parameter	Value
Search method	RandomizedSearchCV
CV folds	3
Search iterations	10
Search sample size	All training prefixes
Scoring	F1-weighted (classification) / RMSE (regression)

4.3 Evaluation Protocol and Comparison Design

We select three metrics for classification. **F1-weighted** balances precision and recall while accounting for class support, making it robust to the 67:33 class imbalance as the primary ranking metric used by the pipeline. **ROC AUC** provides a threshold-independent measure of discrimination as the most commonly reported metric in PPM literature. **MCC** (Matthews Correlation Coefficient) aggregates all four confusion matrix entries into a single score (range -1 to $+1$, where 0 = random) and acts as a balanced tiebreaker when F1 and ROC AUC disagree.

For regression we select **RMSE** in hours (same unit as the target, penalises large errors, primary ranking metric) and R^2 (proportion of variance explained, normalised against the mean baseline).

Every model configuration is trained twice: once with log-only features and once with log-based and time-series features combined. Both configurations share the same train/test split, hyperparameter search space, and evaluation pipeline. The comparison tool (`evaluate_feature_impact.py`) loads both evaluation reports and produces paired metric tables and faceted plots.

5 Results

The best model differs by task: HistGradientBoosting (HGB) achieves the highest F1-weighted for classification (0.6601), while Random Forest (RF) achieves the lowest RMSE for regression (228.97).

5.1 Outcome Prediction

HistGradientBoosting (HGB) outperforms all alternatives for outcome classification, achieving the highest scores across all three primary metrics (Table 10).

HGB leads Random Forest by 2.1% in F1-weighted (0.6601 vs. 0.6465), by 0.006 in ROC AUC, and by 0.019 in MCC.

Table 10: Classification model comparison (log+TS, 58 features, full data).

Model	F1-weighted	ROC AUC	MCC
HistGradientBoosting	**0.6601**	**0.6580**	**0.2316**
Random Forest	0.6465	0.6522	0.2128
Logistic Regression	0.6297	0.6270	0.1827
Dummy (most frequent)	0.5401	0.5000	0.0000

Figure 1 shows F1-weighted across prefix lengths, with performance improving consistently as more events become available: HGB rises from 0.5051 at prefix 3 to 0.7731 at prefix 34. The improvement rate diminishes after the first 10–15 events, but longer prefixes continue to add signal.

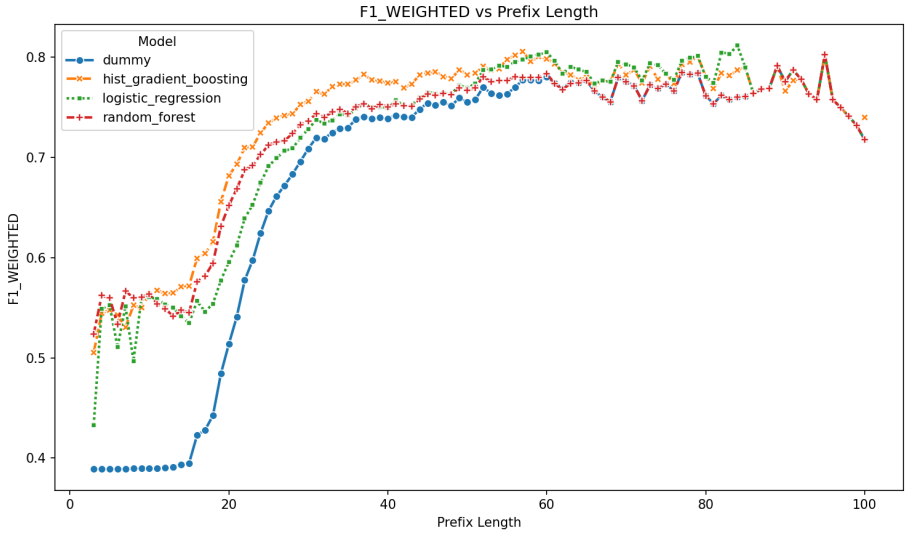

Fig. 1: F1-weighted by prefix length (classification).

Time-Series Impact Adding hourly-resampled TS features to HGB changes nothing. With 34 log-only features, HGB scores 0.6584 F1-weighted and 0.6597 ROC AUC. With 58 log+TS features, it scores 0.6594 and 0.6557 (Table 11). The differences (+0.001 in F1-weighted and −0.004 in ROC AUC) are negligible.

Table 11: Classification TS impact (HGB, full data).

Configuration	Features	F1-weighted	ROC AUC
HGB log-only (34)	34	0.6584	0.6597
HGB log+TS (58)	58	0.6594	0.6557

Feature Importance Table 12 lists the top 10 features for HGB log-only. A single feature dominates: `count_W_Validate application` (importance 0.0636) is three times the second-ranked feature and accounts for the majority of predictive signal.

Table 12: Top 10 features by permutation importance (HGB log-only).

Feature	Importance
count_W_Validate application	0.0636
count_W_Handle leads	0.0209
monthly_cost	0.0123
num_terms	0.0097
elapsed_time_hours	0.0074
count_A_Validating	0.0061
count_O_Returned	0.0060
count_W_Call after offers	0.0057
count_W_Call incomplete files	0.0043
time_since_last_event_hours	0.0023

Figure 2 visualises the importance distribution, with activity-count features occupying 6 of the top 10 positions while financial and temporal features appear at middle ranks with substantially lower importance and waiting-state features appearing nowhere in the top 10.

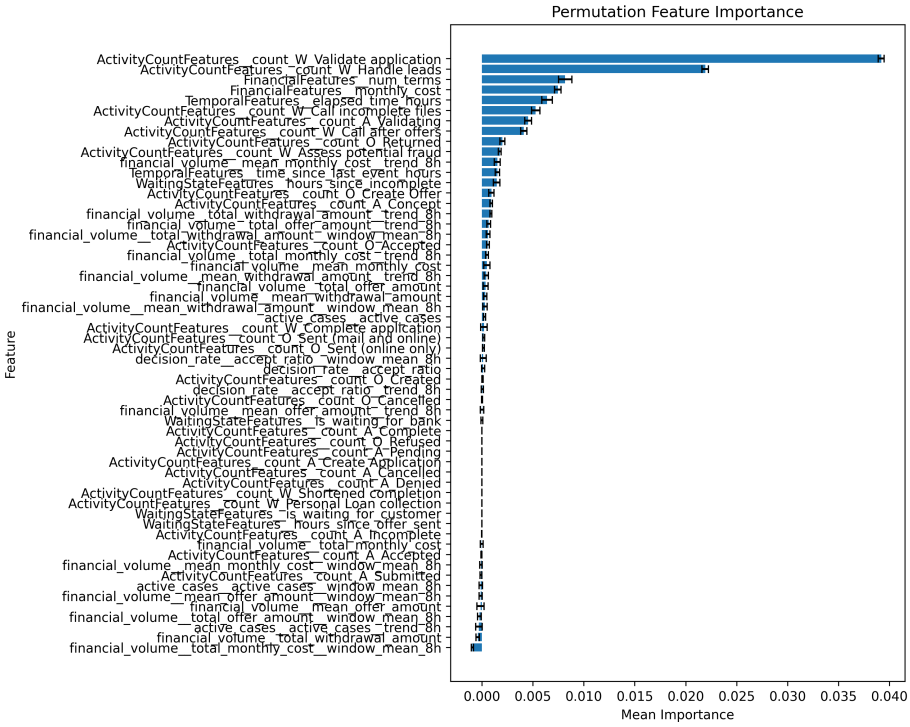

Fig. 2: Permutation feature importance (HGB log-only).

Figure 3 shows the SHAP summary dot plot for the same model, where high occurrences of `count_W_Validate application` push predictions toward approved while higher `elapsed_time_hours` and `time_since_last_event_hours` push toward not-approved, meaning long-running cases end negatively.

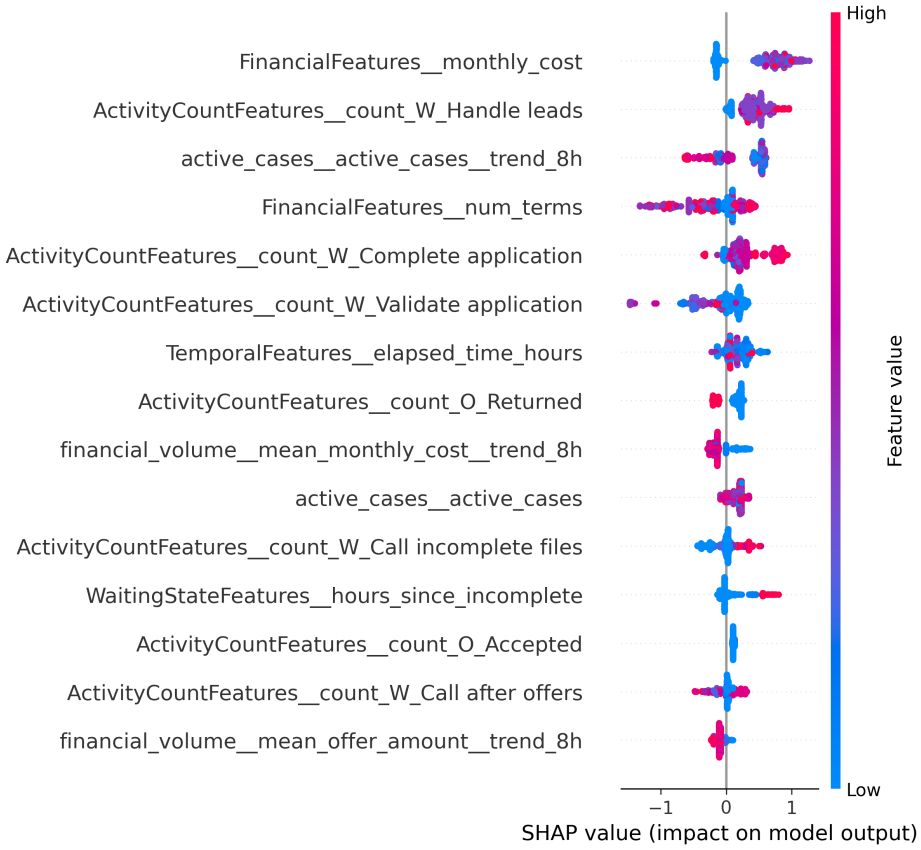

Fig. 3: SHAP summary dot plot (HGB classification). Each dot represents a single prediction. Colour indicates feature value (red = high, blue = low). Features are ordered by global importance.

5.2 Remaining-Time Prediction

Random Forest (RF) achieves the lowest RMSE for remaining-time regression, marginally ahead of HGB and well below the dummy baseline (Table 13). The R^2 of 0.382 indicates that RF explains 38% of the variance in remaining time.

Table 13: Regression model comparison (log+TS, max_length=100, full data).

Model	RMSE (hours)	R^2
Random Forest	**230.28**	0.382
HistGradientBoosting	231.15	–
Ridge	237.33	–
Dummy (mean)	295.97	0.000

Figure 4 shows RMSE across prefix lengths as it improves from 284.83 at prefix 3 to 233.14 at prefix 100, with most of the gain achieved by prefix 20. Beyond that, returns diminish but do not fully plateau.

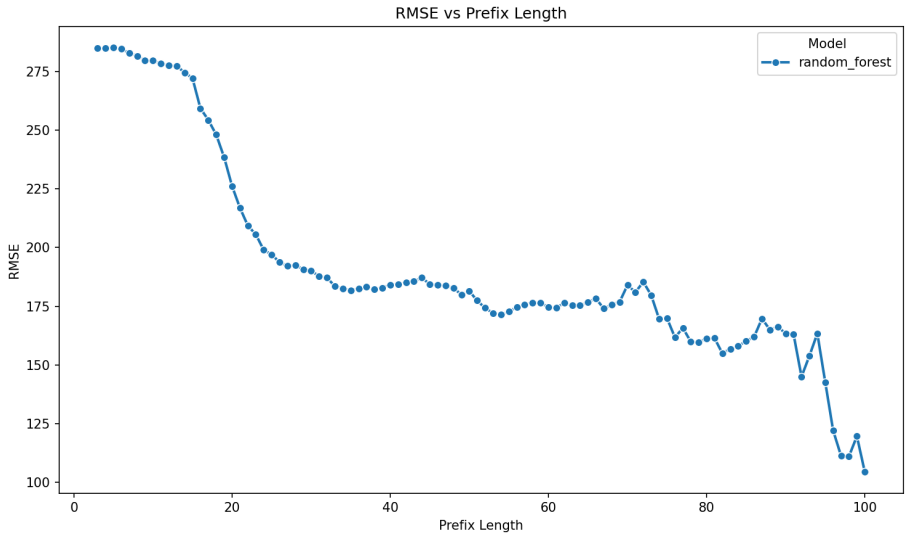

Fig. 4: RMSE by prefix length (RF log-only regression).

Time-Series Impact Time-series features degrade regression performance. The log-only RF (228.97 RMSE) beats every log+TS variant by 1.31–1.45 RMSE (Table 14), a 0.6% penalty. The degradation is consistent across all prefix lengths.

Table 14: Regression TS impact (RF, max_length=100, full data).

Configuration	Features	RMSE (hours)
RF log-only (34)	34	**228.97**
RF log+TS (58)	58	230.42
RF log+TS (compare)	58	230.28

Full-Length vs. Maximum-Length Prefixes Capped (`max_length=100`) and unbounded (full trace) prefix configurations produce nearly identical results at 230.28 vs. 230.20 RMSE, with a difference of 0.04% indicating that longer histories do not help.

Feature Importance Table 15 lists the top 10 features. Top is `count_W_Validate application` (0.0618), then `count_W_Call after offers` (0.0323) and `count_A_Cancelled` (0.0299). Activity counts fill 7 of the top 10. Temporal features are less important.

Table 15: Top 10 features by permutation importance (RF log-only).

Feature	Importance
count_W_Validate application	0.0618
count_W_Call after offers	0.0323
count_A_Cancelled	0.0299
count_A_Validating	0.0132
elapsed_time_hours	0.0131
count_O_Returned	0.0127
count_A_Complete	0.0111
time_since_last_event_hours	0.0096
count_O_Accepted	0.0068
num_terms	0.0048

5.3 Overfitting Analysis

Table 16 reports train-test gaps for the best models while Figure 5 visualises the comparison.

Table 16: Train-test performance gap.

Task	Model	Train	Test (gap)
Classification	HGB (f1-weighted)	0.6971	0.6609 (−0.0362)
Classification	RF (f1-weighted)	0.6810	0.6510 (−0.0300)
Regression	RF log-only (RMSE)	248.08	233.14 (−14.94)

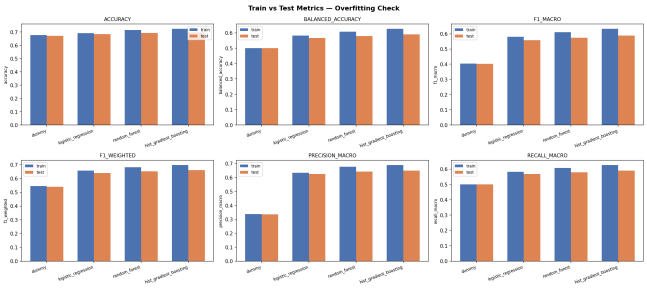

Fig. 5: Train vs. test performance (all models).

Overfitting is moderate with the train-test gap for HGB at −0.036 F1-weighted and for RF regression at −14.94 RMSE (6% of the test RMSE), both within expected bounds for tree-based ensemble models without explicit regularisation.

5.4 Summary of Findings

- **HGB is best for classification** (0.6601 F1-weighted, 0.6580 ROC AUC, 0.2316 MCC), outperforming RF by 2.1%.
- **RF is best for regression** (228.97 RMSE log-only, R^2=0.382), marginally ahead of HGB.
- **Time-series features add no value** for classification (+0.1% F1-weighted) and **harm** regression (+1.45 RMSE).
- **Log-only RF (228.97 RMSE)** beats all log+TS variants, including the HPO-tuned compare run (230.28).
- **Unbounded prefixes yield no improvement** (−0.04% RMSE).
- `count_W_Validate` is the single most predictive feature across both tasks (importance 0.0636 classification, 0.0618 regression).
- Performance improves with longer prefixes, but diminishes after 10–15 events.
- Overfitting gaps are moderate (HGB −0.036, RF −14.94 RMSE).

6 Testing Evidence

The project includes 133 unit and integration tests across 12 test modules:

- **Targets (9)**: Remaining time and outcome label construction on synthetic event logs.
- **Preprocessing (10)**: Sliding-window and outcome-window generation, temporal split integrity.
- **Log-based features (13)**: Activity counts, temporal, financial, and waiting-state features with shape, naming, and edge cases.
- **Time-series features (2)**: Extraction and alignment to prefixes.

- **Dataset (7)**: Download, extraction, caching, and HTTP error handling (1 marked integration).
- **Metrics (33)**: MAE, RMSE, F1, AUC-ROC, R^2, plateau detection, and model comparison correctness.
- **Plots (11)**: Task detection, metric selection, and figure generation.
- **SHAP (5)**: Classification and regression explanations, graceful skip for non-tree models.
- **Trainer (18)**: Model fitting, artifacts, PCA pipeline, and hyperparameter search smoke tests.
- **Config (21)**: Schema validation, estimator construction, YAML round-trip, and error handling.

All tests pass with a single command as follows:

```
poetry run pytest
```

7 Critical Evaluation

7.1 Do Time-Series Features Improve Predictive Performance?

For classification, the hourly-resampled time-series features with 8-hour rolling windows added no meaningful improvement (F1-weighted +0.1%). For regression, they consistently degraded performance (RMSE +1.45), with the log-only model outperforming all log+TS variants across all metric dimensions.

Several factors may explain the limited impact. First, the time-series features were never independently validated as carrying a workload signal: we compute them but do not verify that active-case counts, financial volumes, or decision rates actually track known busy periods. The null result may therefore reflect a poor operationalisation of workload rather than a property of time-series information per se.

Second, a possible **proxy effect**: log-based features (activity counts, elapsed time, case-level financial attributes) may already contain indirect workload information. The dominance of `count_W_Validate application`, a gate activity, across both tasks is consistent with this hypothesis, although it could equally reflect case maturity rather than workload.

Third, several design choices in the extraction template may limit its effectiveness. The **hourly resolution** is coarse: the 8-hour rolling trend (`trend_8h`) is computed as a first-order difference, which amplifies stochastic noise rather than measuring directional change. The alignment step uses backward `merge_asof`, assigning each prefix the most recent past workload value, even though the predictive question is forward-looking as it asks what load the case will experience next. Standard time-series features such as recent lags (t-1h, t-2h) and seasonal lags (t-24h, t-168h) were also omitted despite strong priors for daily and weekly cycles in loan application processing.

Fourth, the **regression degradation** (+1.45 RMSE) may partly reflect the curse of dimensionality: adding 24 features to a 34-feature set increases the search space for tree-based models, and irrelevant features are known to degrade ensemble performance.

7.2 Limitations

Several limitations affect the interpretation of these results. All three time-series signal types (active-case count, financial volume, decision rate) share a single extraction template with hourly binning, 8-hour rolling mean, and 8-hour trend, while other extraction strategies (per-resource load, sub-hourly resolution, seasonal lags) remain unexplored. The 8-hour window covers only 2.6% of the mean case duration (302 hours), with windows at 24, 72, or 168 hours potentially capturing workload seasonality better. The pipeline evaluates Random Forest and HistGradientBoosting as tree-based baselines, since deep learning models (LSTMs, transformers) may integrate continuous time-series information differently than axis-aligned tree splits allow. The temporal split is deterministic, and time-series cross-validation could provide a more rigorous evaluation. No runtime comparison between the log-only and log+TS pipelines was performed, which would inform practitioners of the engineering cost.

All classifiers achieved MCC values below 0.25, indicating modest performance overall. The primary ranking metric F1-weighted is sensitive to class prevalence (67:33 imbalance). Permutation importance, used for feature ranking, is known to be biased when features are correlated, as the time-series rolling features are mechanically correlated with their raw signals and with log-based temporal features, which may affect the importance rankings.

Regarding threats to validity: identical fixed parameters were used for both log-only and log+TS configurations, mitigating concerns that hyperparameter search could favour one feature set. The pipeline is deterministic given fixed random seeds and configuration, ensuring reliability.

8 Conclusion

8.1 Summary

This project investigated whether aligned time-series workload features improve predictive process monitoring. We built a modular four-stage pipeline for the BPI Challenge 2017 loan application log, extracting 34 log-based features and 24 hourly-resampled time-series features with 8-hour rolling windows, while comparing four model families per task. HistGradientBoosting achieved the best classification performance (0.6601 F1-weighted, 0.6580 ROC AUC, 0.2316 MCC), with Random Forest achieving the lowest regression error (228.97 RMSE log-only).

The time-series features added no meaningful value for classification (+0.1% F1-weighted) and degraded regression (+1.45 RMSE), with the log-only RF beating all log+TS variants. The gate activity `count_W_Validate` dominated feature importance across both tasks (0.0636 classification, 0.0618 regression), while unbounded prefix lengths provided no improvement over a cap of 100 events. Overfitting was moderate for both best models.

One possible explanation is a proxy effect, where log-based features may already capture workload variation indirectly through activity counts. The findings

suggest that practitioners can prioritise log-based features, particularly activity counts of gate events, without investing in time-series feature engineering. The pipeline and all experiments are open-source and fully reproducible.

Data and Code Availability

Source code: https://github.com/durki007/spi-time-series

Dataset: The BPI Challenge 2017 log is downloaded automatically on first run.

Reproduction:

1. `pip install -r requirements.txt`
2. `python -m spi_time_series.main experiments/classification/classification_compare.yaml -o results/classification/classification_compare/`
3. `python -m spi_time_series.main experiments/regression/regression_rf_log.yaml -o results/regression/regression_rf_log/`

References

1. van Dongen, B.F.: BPI Challenge 2017. Eindhoven University of Technology (2017). https://research.tue.nl/en/datasets/bpi-challenge-2017

A Appendix: Additional Results

A.1 Classification: Full Model Comparison

Table 17: Classification: full metrics for all models (log+TS, 58 features).

Model	F1-weighted	ROC AUC	MCC
HGB	**0.6601**	**0.6580**	**0.2316**
RF	0.6465	0.6522	0.2128
LR	0.6297	0.6270	0.1827
Dummy	0.5401	0.5000	0.0000

A.2 Regression: Full Model Comparison

Table 18: Regression: full metrics for all models (log+TS, max_length=100).

Model	RMSE	R^2
RF	**230.28**	0.382
HGB	231.15	–
Ridge	237.33	–
Dummy	295.97	0.000

A.3 TS Impact Summary

Table 19: TS impact summary across all configurations.

Task	Model	Features	F1 / RMSE	ROC AUC / R^2
Classification	HGB log-only	34	0.6584	0.6597
Classification	HGB log+TS	58	0.6594	0.6557
Regression	RF log-only	34	**228.97**	0.388
Regression	RF log+TS	58	230.42	0.381
Regression	RF log+TS (full)	58	230.20	0.382

A.4 Overfitting Details

Table 20: Train-test comparison for all models.

Task	Model	Metric	Train	Test
Classification	HGB	F1-weighted	0.6971	0.6609
Classification	RF	F1-weighted	0.6810	0.6510
Classification	LR	F1-weighted	0.6563	0.6388
Classification	Dummy	F1-weighted	0.5454	0.5401
Regression	RF	RMSE	248.08	233.14
Regression	HGB	RMSE	238.74	234.88
Regression	Ridge	RMSE	256.93	240.87
Regression	Dummy	RMSE	295.97	295.97

B Appendix: Configuration Files

Experiments are YAML files under `experiments/{task}/`. Key parameters: `data.split_quantile=0.8`, `prefix.min_length=3`, `search.cv_folds=3`, and `features.enabled_features` selects log-based or time-series feature sets.

C Appendix: Feature Descriptions

C.1 Log-Based Features (34 features)

- **ActivityCountFeatures (26)**: One count per activity (e.g., `count_W_Validate application`, `count_A_Accepted`) in the prefix.
- **TemporalFeatures (2)**: `elapsed_time_hours` (hours since case start) and `time_since_last_event_hours` (hours since the most recent event).
- **FinancialFeatures (2)**: `monthly_cost` and `num_terms` from the loan application's offer.
- **WaitingStateFeatures (4)**: `hours_since_offer_sent`, `hours_since_incomplete`, `is_waiting_for_customer`, `is_waiting_for_bank`.

C.2 Time-Series Features (24 features)

Three columns per signal: raw, 8-hour trailing mean, and 8-hour trend (current minus 8 hours prior).

- **ActiveCaseCountFeature (1 base, 3 total)**: Number of cases active at the prefix's timestamp in the hourly active-case time series.
- **FinancialVolumeFeature (6 base, 18 total)**: Hourly aggregates: total and mean of withdrawal amount, offer amount, and monthly cost.
- **DecisionRateFeature (1 base, 3 total)**: Ratio of accepted decisions (events `O_Accepted`) per hour.